

TERM PROJECT (TAKE HOME FINAL EXAM)

Due date for Reports and Evaluation Paragraph

1 day after the final exam (Ninova, Sunday, 23:59, 11 Jan 2025)

Due date for Submitting Presentations

Just before the final exam (Ninova, Saturday, 14:00, 10 Jan 2025)

In-person Presentation Session

@Final Exam (4104, Saturday, 14:30 – 17:30, 10 Jan 2025)

1. Final (term) projects will be conducted in pre-assigned **teams**. The details of each team's project are explained comprehensively on the **following pages**.
2. If you have **not** previously integrated your robot and sensors into the described Gazebo environment, you must complete this step first. The integration can be completed collaboratively as a team.
3. Follow the steps in the project descriptions to complete term project.
 - a. All comparisons must be performed both **qualitatively** and **quantitatively**.
 - b. Ground truth localization data can be achieved from Gazebo implementing a plugin (e.g. ground_truth/state)
 - c. Prepare a report.
 - d. Prepare a presentation and present it in person at the final exam hours.
 - e. Prepare a video or video set showing your implementations and tests on the defined scenarios. Do not forget to include your video link to your report.
 - f. Prepare an evaluation paragraph (explained below!)
4. Submit your report, presentation, and evaluation paragraph to the Ninova system before the specified date and time.

Report

1. Prepare a report using official IEEE conference template as if preparing a conference paper
 - a. Overleaf template: <https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhnncsfqn>
 - b. Latex/Word templates: <https://www.ieee.org/conferences/publishing/templates>
2. In your project report include the followings:
 - a. Abstract: Summarizing your work and achievements
 - b. Introduction: Problem description, literature survey on this subject, and what is done in this work.
 - c. Methodology:
 - i. a flow chart or block diagram of the project,
 - ii. explanation of the working principles,
 - iii. apart from system integration, an explanation of the code that you developed/wrote for the project (no need to add code to the report, explain how it works, inputs, outputs, parameters, etc.)

- d. Results:
 - i. Simulation results with qualitative and quantitative comparisons
 - e. Media links (e.g. videos showing platform integration to the Gazebo World, project tasks completion)
 - f. Code/repo links (e.g. your GitHub repository)
 - g. References (e.g. papers) cited in the Introduction and the rest of the study, and footnotes (e.g. ROS packages, lecture notes)
3. Your reports should be 6 to 10 pages.

Presentation

1. Prepare a presentation about the project and the results achieved to present face-to-face to the committee on the final exam date.
2. The **agenda** for the presentation day will be announced **later**.
3. Presentation should be 10 pages maximum.
4. Presentation should be done as a team.
5. Presentation should be completed within 10 minutes and after that Q&A session will be held.
6. Add video(s) to the presentation and explain your achievements/missing points on it(them) as well.

Video

1. Prepare a video showing all satisfied requirements in the project description.
2. Add the link of the video(s) to your report.
3. Add your video(s) to your presentation.
4. Video(s) should be edited to make it compact and suitable for presentation. (Hint: You have 10 minutes for presentation!)

Evaluation Paragraph

- Write a paragraph about how the team worked on the project (e.g., team members' efforts, contributions, etc.) and upload it to the Ninova system. If any team member did not contribute, please explicitly state this in your paragraph. (**Compulsory** for each team member and must be uploaded individually!)

PS 1: Any team member can upload the project reports; there is no need for all team members to submit it individually.

PS 2: Please note that if you are not specifically required to use a particular ROS package, you may use any package or write your own code. Utilizing existing ROS packages is permitted.

PS 3: It is strongly recommended to push your project to **GitHub** and include the repository link in your project report.

Team 1

Team Members:

Berke Algül, Mesut Anlak, Mehmet Asan

Robot:

Agilex Limo



Sensors:

1. 2D LiDAR:

270° FOV (2 pieces one for front one for rear), 0.5° resolution with 30m range

2. IMU: $\pm 0.01^\circ$ drift per second

Gazebo World:

AWS Bookstore: <https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps?tab=readme-ov-file#aws-bookstore>

Project Description:

SLAM and Autonomous Navigation

1. Merge 2D LiDAR data to generate single LaserScan message (e.g.

https://github.com/giuschio/scan_merger_2)

2. Find a SLAM tool that has loop closure property and get occupancy grid 2D map of the environment as good as possible

4. Define a global pre-defined path (waypoints) such that robot will move within the entire region

5. Remove some objects in the bookstore like tables, etc and try to navigate autonomously based on predefined global waypoints.

6. Compare localization performance of AMCL w/ and w/o object removal from the environment (get ground truth localization using Gazebo plugins).

7. Add some pedestrians to the environment (e.g.

https://github.com/TempleRAIL/pedsim_ros_with_gazebo) and implement obstacle avoidance based on dynamic objects

8. Investigate AMCL localization performance under dynamic obstacles

Team 2

Team Members:

Samet Atak, Ata Ataş, Ali Baghirov

Robot:

Agilex Hunter 2.0

Sensors:

1. RGBD Camera:

Wide-angle (140° FOV), 6m depth range, 30Hz frame rate

2. 3D LiDAR

Robosense Helios 32

Gazebo World:

Clearpath Robotics Agriculture World:

https://github.com/clearpathrobotics/cpr_gazebo?tab=readme-ov-file#agriculture-world

Project Description:

Multi Sensor 3D SLAM

1. Create Point Cloud Map of the Environment using an algorithm you choose (e.g. <http://wiki.ros.org/octomap>), take wheel odometry as odometry source and use

a) only an RGBD camera-based SLAM

b) only a 3D LiDAR based SLAM

c) both 3D LiDAR and RGBD camera-based SLAM

2. Compare achieved 3D maps in terms of their quality (possible metrics to be used: point cloud density, Peak Signal-to-Noise Ratio (PSNR), SSIM (Structural Similarity Index))



Team 3

Team Members:

Ertuğrul Berber, Ubeydullah Bilgin, Yiğit Çakır

Robot:

Agilex Scout mini

Sensors:

1. 3D LiDAR: VLP-32C
2. IMU: Xsens Mti 300

Gazebo World:

AWS Small House: <https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps?tab=readme-ov-file#aws-small-house>

Project Description:

3D SLAM for Patrol Robot

1. Use Cartographer package or similar to create 3D map of the environment. Combine wheel odometry and IMU data using robot_localization package
2. Add randomly moving pedestrians into the environment (e.g. https://github.com/TempleRAIL/pedsim_ros_with_gazebo).
3. Assign random patrol duty to the robot in the field and design an obstacle avoidance system such that
 - a) wait if any dynamic obstacles on the path until it disappears
 - b) avoid obstacles using local planners by estimating moving direction of the dynamic obstacle
 - c) change the number of dynamic obstacles in the field
 - d) compare performance of the methods quantitatively (e.g. elapsed time, consumed energy, control effort, navigation performance, ...)



Team 4

Team Members:

Ali Taylan Dursun, Ezgi Ekinci, Hamdi Selim Ekşi



Robot:

Agilex Scout 2.0

Sensors:

1. RGBD camera: ZED 2i
2. 2D LiDAR: 180° FOV (2 pieces, one for front one for rear), 0.33° resolution with 15m range

Gazebo World:

Hospital With Two Floors: <https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps?tab=readme-ov-file#hospital-with-two-floors>

Project Description:

Multi Floor Navigation

1. Using 2D SLAM techniques, create 2D occupancy grid map of the floors separately. (You can use, Lidars, camera and wheel odometry data at the same time to improve mapping quality)
2. Add fiducial markers to the Gazebo world floors to inform robot about which floor it is
3. Implement a marker detector to determine which floor the robot is (RGBD camera for detection)
5. Spawn robot in a random place (pose and floor, show at least with 3 trials)
6. Using map_server and custom decision maker (based on marker detector) decide which map will be used for navigation
7. Call global localization service of AMCL to localize robot
 - a) use only one LiDAR for localization
 - b) use both LiDARs for localization
 - c) use LiDARs and RGBD camera for localization
8. Compare global localization performance for different sensor combinations (localization time, accuracy rate, failure rate, etc.)
9. Navigate robot from one point to another (e.g. move_base package in ROS1, nav2 in ROS2)

Team 5

Team Members:

Burak Eren, Semanur Gülmez, Sanem Gümüşlüoğlu

Robot:

Unitree A1



Sensors:

1. 3D LiDAR:

360° horizontal and 30° vertical FOV, 0.5° horizontal and vertical resolution, 50m range, 10Hz refresh rate

2. IMU: $\pm 0.025^\circ$ drift per second

Gazebo World:

Clearpath Robotics Inspection World:

https://github.com/clearpathrobotics/cpr_gazebo?tab=readme-ov-file#inspection-world

Project Description:

3D Elevation Mapping of the Environment

1. Generate Point Cloud Elevation Map for the world (select a proper elevation resolution)
2. Odometry source will be odometry of the robot (if available) + IMU (fusing with robot_localization), otherwise only IMU
3. Write a ROS node that automatically converts the 3D elevation map into 3D occupancy grid map (octomaps) based on elevation level and map resolution selected by user (define them as parameters of the node)
4. Autonomously navigate in this field and show used 3D map represent the environment well for the robot used.

Team 6

Team Members:

Göktuğ Güzel, Eren Harmankaya, Furkan Hayırlıoğlu

Robot:

Clearpath Robotics Husky A200 UGV



Sensors:

1. 3D LiDAR: 360° horizontal and 30° vertical FOV, 0.5° horizontal and vertical resolution, 20m range, 10Hz refresh rate
2. RTK-GPS: $\pm 2\text{cm}$ precision

Gazebo World:

Clearpath Robotics Orchard World:

https://github.com/clearpathrobotics/cpr_gazebo?tab=readme-ov-file#orchard-world

Project Description:

3D Mapping of the Environment

1. Generate Point Cloud Map for the Gazebo environment using RTAB Mapping
2. Odometry source will be RTK-GPS + Wheel odometry of the vehicle (fusing with robot_localization package)
3. Converting point cloud map into octomaps with different resolutions (at least for 3 map resolution cases) and evaluate resolution effects on environment representation
4. Check map resolution effect on autonomous navigation
5. Compare your results qualitatively and quantitatively.

Team 7

Team Members:

Javad Ibrahimli, Kamran Ismayilov, Ali Ulvi Kaplan

Robot:

Robotnik RB-Summit



Sensors:

1. GPS: $\pm 5\text{m}$ accuracy
 2. IMU: Xsens Mti 100
 3. RGB camera
- Flir BlackflyS BFS-U3-122S6C-C: 12.3 MP, 23 FPS

Gazebo World:

AOC Tomato Farm (22mx14m): https://github.com/LCAS/aoc_tomato_farm/tree/humble-dev/tomato_farm_simulator/worlds/22mx14m

Project Description:

Topological Navigation based Inspection Robot

1. Add separate distinguishable planar fiducial markers (e.g., Aruco markers, AprilTags, QR codes) to each row of the farm
2. Create topological map of the environment (manually or with a SLAM method you choose)
 - a. Topological map should define both entrances and center of the rows as nodes (3 nodes per row). Nodes should describe metric position of the nodes with respect to global world frame.
 - b. Define edges between nodes, such that central nodes connected to entrance nodes and entrance nodes are connected to each other.
3. Randomly select one of the markers from the database and search this row with topological navigation framework you created.
4. When the robot reaches the specified row, enter the row and navigate towards the middle node defined in the row.
5. While travelling from entrance to center of the row record video of the camera using a ROS package (e.g. https://wiki.ros.org/image_view). Do not save video while searching the row.

Team 8

Team Members:

Abdullah Kaya, Halil İbrahim Umut Kaya, Muhammed Kılıçaslan



Robot:

Clearpath Robotics Jackal UGV

Sensors:

1. 2D LiDAR:

270° FOV (2 pieces one for front one for rear), 1° resolution with 30m range

2. RGBD camera: Intel Realsense D435i

Gazebo World:

Office Earthquake: https://github.com/leonhartyao/gazebo_models_worlds_collection

Project Description:

Multi Sensor Kalman Filter Localization

1. Try to navigate robot using EKF of robot_localization package with

a) Lidar scan matching odometry (only front lidar) + wheel odometry data

b) Lidar scan matching odometry (both lidars) + wheel odometry data

c) RGBD camera image based visual odometry + wheel odometry data

d) RGBD camera depth data based scan matching odometry + wheel odometry data

e) RGBD camera image based visual odometry + RGBD camera depth data based scan matching odometry

f) RGBD camera image based visual odometry + Lidar scan matching odometry (both lidars)

g) RGBD camera depth data based scan matching odometry + Lidar scan matching odometry (both lidars)

h) RGBD camera image based visual odometry + RGBD camera depth data based scan matching odometry + Lidar scan matching odometry (both lidars)

i) RGBD camera image based visual odometry + RGBD camera depth data based scan matching odometry + Lidar scan matching odometry (both lidars) + wheel odometry combinations.

2. Repeat 1 for UKF of robot_localization package

3. In tests, try to assign covariance parameters based on sensor error rates

4. Compare localization performances with ground truth captured from Gazebo plugins qualitatively and quantitatively (through a metric (e.g. RMSE) on a tabular summary and plots)

5. Add randomly moving pedestrians into the environment (e.g. https://github.com/TempleRAIL/pedsim_ros_with_gazebo). And evaluate how these changes affect the localization performance for RGBD camera used cases.

Team 9

Team Members:

Mert Kılıç, Mustafa Turan Koçak, Kadir Yavuz Kurt



Robot:

Clearpath Robotics Warthog UGV

Sensors:

1. 3D LiDAR: 360° horizontal and 30° vertical FOV, 0.5° horizontal and vertical resolution, 50m range, 10Hz refresh rate
2. RGBD Camera: 120° FOV, 5m depth range, 30Hz frame rate

Gazebo World:

Outdoor: https://github.com/leonhartyao/gazebo_models_worlds_collection

Project Description:

SLAM with Calibration

1. Define pose (position and orientation) of the 3D LiDAR with $\pm 5\%$ error in URDF of the robot, which results in data from Gazebo and model will not be exactly matching. Show this misalignment comparing Gazebo and RViz scenes.
2. Create Point Cloud Map of the Environment using an algorithm you choose (e.g. <http://wiki.ros.org/octomap>) using both 3D LiDAR with mounting pose error and RGBD camera-based SLAM
3. Calibrate 3D LiDAR using a calibration tool in ROS (e.g. https://github.com/ankitdhall/lidar_camera_calibration) and use calibrated tf tree now for SLAM. Compare pre/post calibration effects on misalignment through Gazebo and RViz scenes.
4. Compare achieved 3D maps for both cases in terms of their quality (possible metrics to be used: Peak Signal-to-Noise Ratio (PSNR), SSIM (Structural Similarity Index))

Team 10

Team Members:

Yunus Emre Kuru, Akın Mırık, Muhammet Müftüoğlu

Robot:

Pioneer 3-AT



Sensors:

1. 2D LiDAR:

270° FOV (2 pieces one for front one for rear), 0.5° resolution with 20m range

2. RGB camera: Flir BlackflyS BFS-U3-122S6C-C: 12.3 MP, 23 FPS

Gazebo World:

AWS Office: <https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps?tab=readme-ov-file#aws-office>

Project Description:

Traffic Sign Guided Exploration

1. Add traffic signs (turn left, turn right, go straight, etc.) to the Gazebo world. Robot will follow the instructions there to go from initial point to target.
2. Implement a traffic sign detector using RGB camera (OpenCV or similar packages can be used)
3. Guide robot to move all the ways in the environment to explore the field. Implement a global, local planner, tracker to move the robot (e.g. move_base, nav2)
4. Run SLAM algorithms to create 2D occupancy grid map of the environment
 - a) Hector SLAM
 - b) GMapping
6. Compare results qualitatively and quantitatively
 - a) Output maps
 - b) Localization performance based on ground truth localization obtained from Gazebo plugins

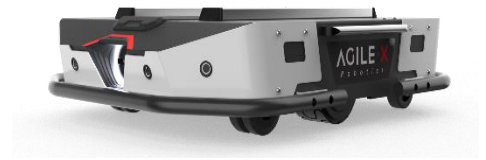
Team 11

Team Members:

Tomasz Naszkowski, Kemal Eren Onay, Yiğit Oturak

Robot:

Agilex Tracer



Sensors:

1. RGBD camera: a ZED 2i and an Intel Realsense D435i (both looking towards forward side of the robot)
2. 3D LiDAR: 360° horizontal and 30° vertical FOV, 0.5° horizontal and vertical resolution, 40m range, 10Hz refresh rate

Gazebo World:

Custom Factory: <https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps?tab=readme-ov-file#custom-factory>

Project Description:

3D SLAM and Global Planner

1. Calibrate (update tf tree for) RGBD cameras using available ROS packages
2. Calibrate RGBD cameras and 3D LiDAR using available ROS packages (e.g. https://github.com/ankitdhall/lidar_camera_calibration)
3. Use RTAB Mapping to get 3D Map of the environment
 - a) Using single camera
 - b) Using both cameras
 - c) Using only LiDAR
 - d) Using cameras and LiDAR
4. Compare output map quality (Map density, similarity to real environment)
5. Assume that your robot will be used for logistics purpose in the field. Design a global planner to cover entire free space of the environment based on 3D map. Consider 3D collusions in the inflated cost map generation for path planning (e.g. voxel inflation).

Team 12

Team Members:

Birkan Özkan, Umut Özkan, Diego Borrajo Ramos

Robot:

Neobotix Neo-MP-500



Sensors:

1. 2D LiDAR: 180° FOV (2 pieces one for front one for rear), 1° resolution with 20m range
2. IMU: $\pm 0.015^\circ$ drift per second

Gazebo World:

Office Env Large: https://github.com/leonhartyao/gazebo_models_worlds_collection

Project Description:

Exploration and Autonomous Navigation

1. Explore the environment as fast as possible

a) Implement https://wiki.ros.org/explore_lite for exploration or similar

i) Single Lidar

ii) Multi Lidar

b) Manual drive by user for exploration

i) Single Lidar

ii) Multi Lidar

c) Compare mapping performance of four cases qualitatively and quantitatively in terms of elapsed time, map quality, etc.

3. Navigate robot autonomously using obtained map (with move_base, nav2, etc.) and use robot_localization package for sensor fusion

4. Try to dock the robot to a target with high accuracy (less than 1 cm error to assigned target)

a) Did your setup manage to dock to the target?

b) How to improve the planning and tracking performance?

c) Try to improve the docking performance of the robot.

Team 13

Team Members:

Tunahan Şahin, Muhammet Ali Sanduvaç, Abdullah Soliman

Robot:

Clearpath Ridgeback



Sensors:

1. 2D LiDAR:

240° FOV (2 pieces one for front one for rear), 1° resolution with 20m range

2. IMU: $\pm 0.015^\circ$ drift per second

Gazebo World:

Warehouse: https://github.com/belal-ibrahim/dynamic_logistics_warehouse

Project Description:

Multi Map Based Navigation

1. Use robot_localization package to fuse wheel odometry and IMU data and use fused odometry in SLAM
2. Merge 2D LiDAR data and increase resolution for the overlapping field of views.
3. Using 2D SLAM techniques obtain 2D occupancy grid map of the environment
 - a) single map for entire field
 - b) multiple maps (at least three) with Multi map SLAM and connect them using a topological map
4. Autonomously navigate robot between submaps
5. Compare mapping and navigation performance of both approaches using some metrics (e.g. localization accuracy comparisons with ground truth achieved from Gazebo plugin, map quality, AMCL localization uncertainty levels)

Team 14

Team Members:

Ceylan Tolunay, Atakan Yaman, Eren Yüçetürk

Robot:

Pioneer 3-DX



Sensors:

1. RGBD Camera:

90° FOV, 4m depth range, 30Hz frame rate

2. IMU:

$\pm 0.1^\circ$ drift per second

Gazebo World:

Clearpath Robotics Office World:

https://github.com/clearpathrobotics/cpr_gazebo?tab=readme-ov-file#office-world

Project Description:

3D SLAM and Autonomous Navigation

1. Use robot_localization package to fuse IMU and wheel odometry data.
2. Convert (if needed) the depth data of the RGBD camera to PointCloud2 message.
3. Use faster_lio SLAM package (<https://github.com/gaoxiang12/faster-lio>) or similar for 3D map building
4. Use fast_lio SLAM package (https://github.com/hku-mars/FAST_LIO) or similar for 3D map building
5. Compare localization performance of the lidar inertial odometry outputs of the frameworks with ground truth achieved by gazebo plugin
6. Compare 3D mapping performance of the methods qualitatively and quantitatively (e.g. point cloud density)
7. Use 2D projection of the computed 3D map for navigation. Assign random points in the environment to move the robot autonomously (e.g. move_base, nav2 packages for navigation)